Design-for-Testability in Reversible Logic Circuits based on Bit-Swapping

Joyati Mondal*, Debesh K. Das*, Bhargab B. Bhattacharya†
*Dept. of Computer Science & Engg., Jadavpur University, Jadavpur, Kolkata - 700 032, India
†ACM Unit, Indian Statistical Institute, Kolkata - 700 108, India
joyatimondal14@gmail.com, debeshd@hotmail.com, bhargab.bhatta@gmail.com

Abstract*— The emerging technology of reversible circuits offers a potential solution to the synthesis of ultra low-power quantum computing systems. A reversible circuit can be envisaged as a cascade of reversible gates only, such as Toffoli gate, which has two components: k control bits and a target bit (k-CNOT), $k \geq 1$. While analyzing testability issues in a reversible circuit, the missing-gate fault model is often used for modeling physical defects in quantum k-CNOT gates. In this paper, we propose a new design-for-testability (DFT) technique for quantum reversible circuits that deploys bit-swapping using Fredkin gates. It is shown that in an $(n \times n)$ circuit implemented with k-CNOT gates, addition of only two extra inputs along with a few Fredkin gates yields easy testability in the circuit. The modified design admits a universal test set of size $(n + k + 2)$ that detects all detectable missing gate faults in the original circuit, where k is the maximum number of controls used among all k-CNOT gates. The DFT overhead in terms of quantum cost is also much less compared to previous approaches.

***Index Terms*—Missing-gate fault, quantum computing, reversible logic, test generation**

I. Introduction

Concern for energy loss has become a fundamental problem in modern nano-scale CMOS logic circuits. According to Landauer's principle [1], a non-reversible circuit causes irreversibility of information processing with which, energy loss is associated. Reversible logic, on the other hand, provides a mechanism for designing information lossless circuits, and is likely to provide a solution to energy reduction [1]–[3]. It also provides a foundation for designing quantum computers [4], [5].

Classical logic gates such as AND, OR, NAND, NOR, or XOR used in digital design are not reversible, and hence, they cannot be used to design reversible circuits. In order to design reversible circuits, only reversible gates can be used such as controlled-not (CNOT) gate proposed by Feynman [6], Toffoli [7], or Fredkin [8] gates, among others.

Several researchers [9]–[12] have studied the problem of fault modelling and testing of reversible logic circuits. While the stuck-at fault model is widely used for conventional logic circuits, newer models such as single missing-gate fault (SMGF), repeated-gate fault (RGF), partial missing-gate fault (PMGF), or multiple missing gate fault (MMGF), have been found to be more suitable for representing physical failures or defects in reversible circuits implemented with quantum technologies [9], [11], [12].

In this paper, we propose a testable design for a given reversible circuit based on bit-swapping technique. We make an $(n \times n)$ reversible circuit implemented with k-CNOT gates easily testable by adding only two extra input lines and some extra Fredkin gates so that all SMGFs, RGFs, PMGFs, and MMGF's are detected by a universal test set of size $(n + k + 2)$, where k is the maximum number of controls used among all k-CNOT gates. Previous DFT methods [10] [13] used for achieving universal testability in a reversible circuit led to very high quantum cost. In contrast, the proposed method, as demonstrated by experimental results on large benchmark circuits, provides easy testability with much less quantum cost.

II. Preliminaries

A. Reversible logic

A reversible function has equal number of inputs and outputs, and the function simply induces a permutation on the set of input vectors to produce an output vector. There is a bijection mapping between inputs and outputs. Further, in a circuit implemented with reversible gates, no fanout is allowed.

B. Reversible gates

The basic CNOT-type reversible gates used for synthesis are the following:
(i) $(1 \times 1) NOT(x_1 \rightarrow x_1')$
(ii) (2×2) controlled NOT (CNOT):$(x_1, x_2) \rightarrow (x_1, x_1 \oplus x_2)$
(iii) (3×3) Toffoli gate $(x_1, x_2, x_3) \rightarrow (x_1, x_2, x_1 x_2 \oplus x_3)$.
A generalized Toffoli gate, also known as MCT gate [8], has a set of control inputs *C*, a target input set *T*, and has the form *TOF(C;T)*, where $C = (x_{i1}, x_{i2}, ..., x_{ik})$ $T = \{x_j\}$ and $C \cap T = \phi$. It maps an input vector $(x_1^0, x_2^0, ..., x_n^0)$ to $(x_1^0, x_2^0, ..., x_{j-1}^0, x_j^0 \oplus (x_{i1}^0.x_{i2}^0.....x_{ik}^0), x_{j+1}^0, ..., x_n^0)$. Thus, a NOT gate is $TOF(x_j)$, i.e., a generalized Toffoli gate which has no controls. The CNOT gate is $TOF(x_i, x_j)$, i.e., a generalized Toffoli gate with one control bit [9]; this is also known as Feynman gate. The simple (3×3) Toffoli gate is a generalized Toffoli gate with two controls [16]. A k-CNOT gate has *k* control inputs $x_1, x_2, ..., x_k$ and one target input t. It maps the input vector $(x_1, x_2, ..., x_k, t)$ to the output vector $(x_1, x_2, ..., x_k, t \oplus x_1.x_2...x_k)$. Fig. 1 shows the standard graphic symbol for some common CNOT-based reversible gates.

978-1-4673-9739-1/15 $31.00 © 2015 IEEE
DOI 10.1109/ATS.2015.47

IEEE computer society

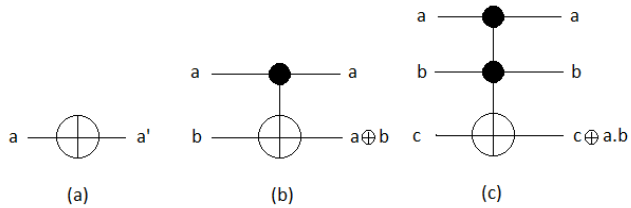

Fig. 1: Different types of CNOT-based reversible gates: (a) NOT gate (b) CNOT gate (c) Toffoli gate

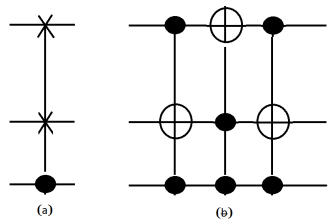

Fig. 2: (a) Fredkin gate Symbol (b) implementation using Toffoli gates.

C. Fredkin gates

A multiple-control Fredkin gate (MCF) [8] with target lines x_p and x_q, and k control lines $x_{i_1}, x_{i_2}, ..., x_{i_k}$, maps $(x_1x_2...x_p...x_q...x_n)$ to $(x_1x_2...x_q...x_p...x_n)$ if all the control lines have value 1. An MCF gate is thus a controlled-swap of target lines. An MCF gate, with no control, always swaps the target lines. The gate with a single control line is the original gate defined by Fredkin.

An MCF gate is denoted as $MCF(C; t_p, t_q)$, where C is the possibly empty set of control lines, and t_p and t_q are target lines. Note that the control lines and unconnected lines pass through both types of gates unchanged. We follow the convention of using the symbol $\bigoplus$ to denote the target of an MCT gate, $\times$ to denote the targets of an MCF gate, and solid black circles to indicate control connections.

A single-control $MCF(t_p, t_q, x_i)$ can be implemented by three 2-CNOT gates. The symbol of a Fredkin gate and its implementation using 2-CNOT gates are shown in Fig. 2(a) and Fig. 2(b), respectively.

D. Quantum cost analysis

The quantum cost of a gate G is the number of elementary quantum operations required to realize the function given by G [14]. Table I shows the required quantum cost $Q_c(k)$ in the realization of Toffoli gate or MCT gate with k controls [14]. An MCT gate with $(k+1)$ lines has k control lines.

controls (k)	0	1	2	3	4	5	6	7	8
Quantum Cost $(Q_c(k))$	1	1	5	13	29	61	125	253	509

TABLE I: Quantum cost of a Toffoli gate with n controls

The quantum cost of an MCF (Multiple-control Fredkin gate) with m control and target lines is calculated as the cost for an MCT gate with the given number of lines plus 2 [15].

Thus, the quantum cost of a single-control MCF gate is 7.

E. Fault detection in Toffoli gates

Several fault models for *k*-CNOT based reversible circuits were introduced earlier [9], [11], [12] as described below.

1) Single Missing Gate Fault (SMGF): A *single missing-gate fault* (SMGF) causes the removal of one *k*-CNOT gate from the circuit. Fig. 3(b) shows an SMGF in the circuit shown in Fig. 3(a). The box in Fig. 3(b) marks the position of the missing gate in the fault-free circuit. An SMGF is detected by an input vector that justifies logical 1 at all of the gate's control inputs, and sets either 0 or 1 on the target input, as well as on all wires that are not connected to the gate. In the example of Fig. 3(b), if we apply (1,0,1,1) at the input of the circuit, the normal output in Fig. 3(a) would be (1,1,0,1), whereas in the presence of SMGF fault marked by the dotted box, the output in Fig. 3(b) will be (1,0,1,1).

2) Models derived from single missing-gate fault: A number of fault models have been derived from SMGFs. These include the repeated-gate fault model, the multiple missing-gate fault model, and the partial missing-gate fault model [9].

The *repeated-gate fault* (RGF) model assumes that a gate in a circuit is repeated, i.e., shows up several times consecutively. It is easy to see that if the number of occurrences is even, an RGF's effect is identical to that of an SMGF at the same gate, and it does not alter the function of the circuit [14]. Hence, the RGF model is subsumed by the SMGF model. Fig. 3(c) shows an example, where the first gate is repeated in the circuit shown in Fig. 3(a). If we apply (1,0,1,1), the normal output would be (1,1,0,1), whereas in the presence of the the above RGF marked by the box, the output will be (1,0,1,1). Hence, it is detected by the vector (1,0,1,1).

The *multiple missing-gate fault* (MMGF) model assumes that several consecutive gates are missing [9]. A complete SMGF test is not guaranteed to cover all MMGFs or multiple SMGFs [12]. Fig. 3(d) shows a MMGF in which the position of the consecutive missing gates are shown by a box. If we apply (1,0,1,1), the normal output would be (1,1,0,1), whereas in the presence of above MMGF, the output will be (1,0,1,1). Hence, it is detected by the vector (1,0,1,1).

A *partial missing-gate fault* (PMGF) [9] corresponds to removing *m* out of *k* control inputs in a *k*-CNOT gate, thus transforming it into a (*k*-*m*)-CNOT gate. The number *m* is called the *order* of the PMGF. Fig. 3(e) shows an PMGF with a box marking the position of the missing control.

In this circuit, if we apply (1,0,0,1), the normal output would be (1,0,0,1), whereas in the presence of the first-order PMGF fault as shown, the output will be (1,1,1,0). Hence, the vector (1,0,0,1) detects this fault. For a gate with *k* control lines, a first-order PMGF is detected by setting missing control line to 0, and setting 1's to all other control lines.

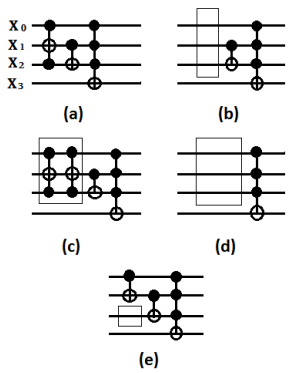

Fig. 3: (a) Fault-free circuit; (b) the same circuit with a SMGF, (c) with an RGF, (d) with an MMGF, (e) with a PMGF

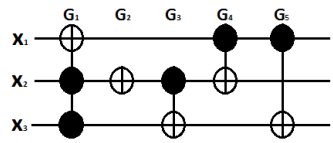

Fig. 4: A reversible circuit

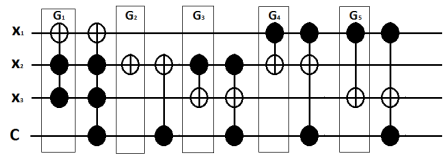

Fig. 5: DFT for the circuit of Fig. 4 following a previous technique [10]

All k first-order PGMFs of a gate have detection conditions that conflict with each other and with the SMGF at the same gate; hence, k+1 test vectors are required to test for all faults. However, these vectors are guaranteed to cover all higher-order PMGFs at the same gate [10].

F. Fault detection in Fredkin gates

No fault model for Fredkin gates has been studied so far. In order to detect faults in a MCF gate with a single control, we are using its equivalent circuit as shown in Fig. 2(b). If we consider all missing-gate faults in Fig. 2(b), the test set is given by {{0,1,1},{1,0,1},{0,0,1},{0,1,0},{1,0,0}}.

III. Earlier Work

Several DFT solutions using Universal Test Sets to detect SMGFs, PMGFs, and detectable RGFs were reported earlier [10], [13].

A. DFT by Rahaman et al. [10]

In this method, the given circuit is first augmented with an additional input C, and then every gate is duplicated using an extra control on this input. During testing phase, C is set to logic 1, and during normal operation, it is set to 0. The latter condition makes all additional gates non-functional, thus retaining the functionality of the original circuit.

The augmented circuit resulting from the method [10] is shown in Fig. 5 for the original circuit of Fig. 4. The gates of Fig. 4 are shown in boxes in Fig. 5. Since with the increase in control lines, the quantum cost of a gate rises exponentially, the overhead incurred in this technique is very large. Moreover, the design does not provide 100% PMGF fault coverage.

B. DFT by Mondal et al. [13]

The DFT approach proposed by Mondal et al. [13] eliminates some of the drawbacks of previous work [10].

In this DFT design, two additional input lines C_1 and C_2 are needed. For each gate G, some additional gates are inserted so that the same test set T is applied to the next gate of G. This method guarantees 100% PMGF fault detection. The quantum cost is decreased by the following method described below.

There may be three cases when G is not the last gate.
Let l = the number of control lines of gate G.
Case 1: $l = 0$;
In this case, G is duplicated in the same way as in [10] to achieve DFT.

Case 2: $l \neq 0$ and $Con(G) \leq \lfloor n/2 \rfloor$, where n is the size of the original circuit, and $Con(G)$ is the number of control inputs of gate G.
For any gate G ($TOF(C,T)$) of Fig. 6(a) with control in the original circuit, a set of 4 k-CNOT gates $TOF(T,C_2)$ as G_1, $TOF(T,C_2)$ as G_2, $TOF(C_1,C_2,T)$ as G_3 and $TOF(C,C_2)$ as G_4 are inserted, as shown in Fig. 6(b). Let G^* be the last gate before G having one or more control bits. When G^* have control and G has target on the same line, G_1 is omitted. Similarly, when G^* has a target and G has control on the same line, where G^* is the next gate following G having one or more control, G_4 is omitted.

Case 3: $l \neq 0$ and $Con(G) > \lfloor n/2 \rfloor$.
For any gate G ($TOF(C,T)$) of Fig. 6(a) with control in

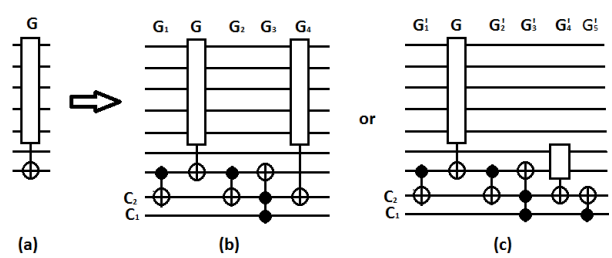

Fig. 6: Testing of gate G (Fig. 6(a)) in the proposed DFT design for Case 2 (Fig. 6(b)), and Case 3 (Fig. 6(c)).

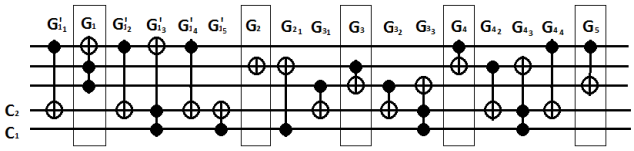

Fig. 7: Testable design for the reversible circuit of Fig. 4 (gates of Fig. 4 are shown in boxes)

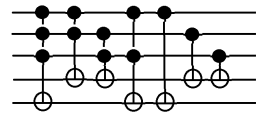

Fig. 8: A reversible circuit with separate control and target lines.

the original circuit, a set of 5 k-CNOT gates $TOF(T, C_2)$ as $G_1^{'}$, $TOF(T, C_2^{'})$ as G_2, $TOF(C_1, C_2, T)$ as $G_3^{'}$, $TOF(n-C, C_2)$ as $G_4^{'}$, and $TOF(C_1, C_2)$ as $G_5^{'}$ are inserted as shown in Fig. 6(c) . Let G^* be the last gate before G having one or more control. When G^* has control and G has target on the same line, $G_1^{'}$ is omitted. The DFT Design for the circuit of Fig. 4 obtained by following this method is shown in Fig. 7.

IV. Proposed Testable design

A major drawback of previous DFT designs is that they incur very high quantum-cost overhead. The situation becomes worse for large circuits. The design proposed here aims to provide a DFT solution with reduced quantum cost that supports a universal test set for the detection of all missing gate faults.

Lemma 4.1: [13] A universal test set for detecting all SMGFs and all PMGFs in a general $(n \times n)$ reversible circuit, must include at least n test vectors.

Suppose a gate G requires a test set T to detect all faults. If one of the control lines in the next gate uses the target line of gate G, then the next gate may not receive the desired test.

For some special circuits, where a gate is not using any control line fed by the target of the previous gate, we have an advantage in testing. Fig. 8 shows an example of these type of circuits. This circuit does not require DFT, and the following test set is sufficient.

$$\begin{pmatrix} 0 & 1 & 1 & X & X \\ 1 & 0 & 1 & X & X \\ 1 & 1 & 0 & X & X \\ 1 & 1 & 1 & X & X \end{pmatrix}$$

Note that in the circuit of Fig. 8 the control lines and target lines are separate. However, in general, such a structure may not always be present.

Given a general reversible circuit, we want to redesign it such that the control lines and target lines are separated. We can easily achieve this by swapping the lines using Fredkin gates.

In the proposed DFT design, we use two additional input lines C_1 and C_2. Before each gate we insert Fredkin gates whenever necessary. We may skip the insertion step when G is the last gate in the original circuit.

For an $(n \times n)$ circuit G, we first locate the largest-size gate, i.e., the one with the maximum number of control inputs. Suppose this number is k $(0 \leq k \leq n-1)$, and let x be the number of lines used as target in the circuit $(1 \leq x \leq n)$. The following algorithm scans the whole circuit and determines k and x.

Algorithm: To find the maximum-size control (k) of a gate and the number of lines with target (x) in a reversible circuit.
Input: A reversible circuit G with size n and depth N;
Output: The maximum-size control k and the number of lines which contains x target lines in a gate G.

BEGIN
1. k=0, $target[1..n]$=0,..,0.
2. **for** i = 1 to N
3. $temp$=0
4. **For** j=0 to n
5. **if** (gate G_i has control in input j)
6. $temp = temp + 1$;
7. **if** (gate G_i has target in input j)
8. $target[j]$=1;
9. **End For**.
10. **if** ($temp > k$)
11. k=$temp$.
12. **End For**
13. $temp$=0
14. **for** i = 1 to m
15. if ($target[j]$==1)
16. $temp = temp + 1$;
17. **End For**
18. x=$temp$;

END.

Depending on the value of x and k, two different approaches are adopted for achieving DFT.

Case 1 : if ($k \leq x$ AND $k + x = n$);
For every gate G_i that has a control on any of the target lines m (i.e., the input lines on which a target appears) ranging from 1 to k, we place a controlled-swap gate MCF(j, m, C_1) with target in j^{th}, and m^{th} input lines; also place a control C_2 before the gate that transfers the control of G_i from m to j, where j is the first input line having no control in G_i. For a 1-CNOT gate, an extra control on line C_1 is added to the gate.

Case 2 : if ($k > x$ OR $k + x \neq n$);
For every gate G_i that has a control on any of the input lines m ranging from $k+1$ to n, we place a controlled-swap gate with target on j^{th}, and m^{th} input lines; also place a control C_2 before the gate that transfers the control of G_i from m to j, where j is the first input line having no control in G_i. Similarly, we check k input lines for target. If any target exists among line m, where $m < k$, we place a controlled

Fredkin gate MCF(m, y, C_1) with the target on line m, and any line y between $k+1$ and $n-1$. The target in the gate moves from x to y. Note that all swap gates are placed before a gate with a modified position for control and target. For an 1-CNOT gate, an extra control on line C_1 is added to the gate.

The algorithm is stated as follows.

Algorithm: DFT
Input: A reversible circuit G with size n and depth N;
Output: Testable Design for reversible Circuit G.
BEGIN
For j=0 to $n-1$
  $Input[j] = j$
if ($k \leq x$ AND $k + x = n$)
  For i=0 to $N-1$
    $m = 0$.
    For j=$n-1$ to 0
      if (G_i has control on j and j is a target line)
        while(G_i has control on m)
          $m = m + 1$;
        $swap(j, m)$
        Interchange $Input[j]$ and $Input[m]$.
    for j=0 to $n-1$
      if (G_i has control/target on j)
        For k = 0 to $n-1$
          if ($Input[k]$==j)
            Put control/target in k
    if (G_i has only one control)
      add a extra control to it in line C_1.
End if
Else
  For i=0 to $N-1$
    $m = 0$.
    For j=$n-1$ to k
      if (G_i has control on j)
        while(G_i has control on m)
          $m = m + 1$;
        $swap(j, m)$
        Interchange $Input[j]$ and $Input[m]$.
    for j=0 to $k-1$
      if (G_i has target on j)
        $swap(j, n-1)$
        interchange $Input[j]$ and $Input[k-1]$
    for j=0 to $n-1$
      if (G_i has control/target on j)
        For k = 0 to $n-1$
          if ($Input[k] == j$)
            Put control/target in k
    if (G_i has only one control)
      add an extra control line C_1 with it.
End else
End

Lemma 4.2: The following test set of size $k+1$ detects all missing gates faults of the original gates, when the augmented circuit is used.

$$S_1 = \begin{pmatrix} C_1 & x_1 & x_2 & \cdots & x_k & x_{k+1} & \cdots & x_n & C_2 \\ 0 & 1 & 1 & \cdots & 1 & 0 & \cdots & 0 & 0 \\ 1 & 0 & 1 & \cdots & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & 1 & 1 & \cdots & 0 & 0 & \cdots & 0 & 0 \end{pmatrix}$$

Proof : The input lines x_1 to x_k consist of all control lines of the original circuit. These lines do not include the target of any Toffoli gate. When the above test set is applied as input to the control lines, it detects all missing-gate faults. Since these lines do not include any targets, the test set remains unmodified and can be fed as inputs to all gates. The test patterns set logic 0 to C_2, and hence, during the propagation of these test patterns, the Fredkin gates remain inactive.

Thus, the above test set of size $k+1$ detects all missing gate faults in the original circuit. ∎

Lemma 4.3: The following test set of size $n+1$ is sufficient to test faults in all Fredkin gates.

$$S_2 = \begin{pmatrix} x_1 & x_2 & \cdots & x_k & x_{k+1} & \cdots & x_n & C_1 & C_2 \\ 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & 0 & 1 \\ 0 & 1 & \cdots & 0 & 0 & \cdots & 0 & 0 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 1 & 0 & 1 \\ \\ 0 & 0 & \cdots & 0 & 1 & \cdots & 1 & 0 & 0 \\ 1 & 1 & \cdots & 0 & 0 & \cdots & 0 & 1 & 0 \end{pmatrix}$$

Proof : In order to test a Fredkin gate MCF(x_p,x_q,C_2), we require the test set {{0,1,0},{1,0,0}, {0,1,1},{1,0,1},{0,0,1}} comprising five tests that detect all missing gate faults. The first n test patterns of S_2 produce tests {0,1,1},{1,0,1},{0,0,1}} at the inputs to the Fredkin gates. The $(n+1)$th test of S_2 produces (0,1,0) at the input to Fredkin gates. The last vector of S_2 produces (1,0,0) at the input to Fredkin gates. Thus, all the required test vectors for Fredkin gates can be applied. It can also be shown that if Fredkin gate is faulty, its effect can be propagated to the output. ∎

Notice that the last vector of S_1 and S_2 are identical.

Theorem 4.4: The modified circuit derived by the proposed DFT method admits the following universal test set of length $n+k+2$.

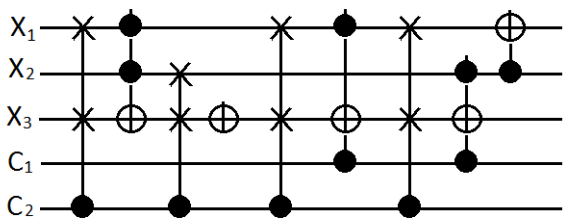

Fig. 9: The proposed testable design for the circuit of Fig. 4.

$$S_U = \begin{pmatrix} C_1 & x_1 & x_2 & \cdots & x_k & x_{k+1} & \cdots & x_n & C_2 \\ 0 & 1 & 1 & \cdots & 1 & 0 & \cdots & 0 & 0 \\ 1 & 0 & 1 & \cdots & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & 1 & 1 & \cdots & 0 & 0 & \cdots & 0 & 0 \\ \\ 0 & 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & 1 & \cdots & 0 & 0 & \cdots & 0 & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & 0 & \cdots & 1 & 1 \\ \\ 0 & 0 & 0 & \cdots & 0 & 1 & \cdots & 1 & 0 \end{pmatrix}$$

Proof : Follows from $Lemma$ 4.2 and $Lemma$ 4.3. ∎

Example 1: The testable design obtained by applying the DFT algorithm for the circuit of Fig. 4 is shown in Fig. 9. The circuit shown in Fig. 9 can be tested by the test set:

$$S_U = \begin{pmatrix} C_1 & x_1 & x_2 & x_3 & C_2 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

V. Experimntal Results

The proposed DFT method is applied to several reversible benchmark circuits (http://webhome.cs.uvic.ca/ dmaslov/). We have considered a few large circuits, the results of which are shown in Table 2. Columns 1, 2, 3 and 4 denote the circuit name, input, size, gate count and quantum cost, respectively. The quantum cost incurred by two earlier DFT methods [10], [13], and by the proposed technique, are shown in columns 6, 7 and 8, respectively. Experimental results indicate that the quantum cost is reduced significantly when the proposed DFT scheme is adopted.

Circuit	size	depth	Qquantum cost	No of swap gates required	% increment in Q.C		
					DFT[[10]]	DFT[[13]]	Our design
cycle10_2d1	12	19	1198	27	204.76	58.18	16.61
ham15d1	15	132	1827	72	221.67	150.57	27.54
cycle17_3d1	20	48	6057	77	202.38	197.62	9.15
gf2$\hat{1}$4mult	42	235	1019	84	269.19	261.43	57.70
mod1024adder	20	55	1541	141	210.70	124.98	64
5mod5d1	6	17	185	0	227.56	165.95	0
5mod5d5	6	7	429	0	204.89	113.05	0
permanent3x3	12	30	1872	0	204.80	112.82	0
permanent2x2	6	3	39	0	223.08	161.53	0

TABLE II: Quantum-cost overhead of different DFT designs.

VI. Conclusion

We have proposed a novel DFT technique for reversible circuits to make them easily testable with a universal test set of length $(n + k + 2)$, where n is the number of input lines, and k is the size of maximum control in the circuit. Our design is found to be best suited for large-sized circuits. For a number of benchmark examples, we have observed a significant reduction in quantum cost compared to previous approaches.

References

[1] R. Landauer, "Irreversibility and heat generation in the computing process," *IBM Journal of Research and Development*, vol. 5, pp. 183–191, 1961.

[2] C. H. Bennett, "Logical reversibility of computation," in *IBM J. Research and Development*, vol. 17, 1973, pp. 525–532.

[3] R. P. Feynman, *Feynman Lectures on Computation*, paperback ed. Addison-Wesley, 1996.

[4] M. Nielsen and I. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.

[5] N. Gershenfeld and I. L. Chuang, "Quantum computing with molecules," in *Proc. of Scientific American*, June 1998.

[6] R. Feynman, "Quantum mechanical computers," in *Optic News*, vol. 11(2), 1985, pp. 11–20.

[7] T. Toffoli, "Reversible computing," MIT Lab for Computer Science, Technical memo MIT/LCS/TM-151, February 1980.

[8] E. Fredkin and T. Toffoli, "Conservative logic," *International Journal of Theoretical Physics*, vol. 21, pp. 219–253, 1982.

[9] J. P. Hayes, I. Polian, and B. Becker, "Testing for missing-gate faults in reversible circuits," in *Proc. of Asian Test Symposium (ATS 2004)*, 2004, pp. 100–105.

[10] H. Rahaman, D. Kole, D. Das, and B. Bhattacharya, "On the detection of missing-gate faults in reversible circuits by a universal test set," in *Proc. of International Conf. on VLSI Design*, 2008, pp. 163–168.

[11] I. Polian, J. P. Hayes, T. Fienn, and B. Becker, "A family of logical fault models for reversible circuits," in *Proc. of Asian Test Symposium*, 2005, pp. 422–427.

[12] ——, "Advanced modeling of faults in reversible circuits," in *Proc. of 8th East-West Design and Test Symposium (EWDTS 2010)*, 2010, pp. 376–381.

[13] J. Mondal, D. K. Das, D. K. Kole, H. Rahaman, and B. B. Bhattacharya, "On designing testable reversible circuits using gate duplication," in *Proc. of VDAT*, 2013, pp. 322–329.

[14] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVinchenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," in *The American Physical Society*, 1995, pp. 52:3457–3467.

[15] D. M. Miller, D. Willie, and R. Drechsler, "Reducing reversible circuit cost by additional line," in *Proc. of ISMVL*, June 2015, pp. 318–323.